

Programación y Algoritmos I – Tarea 10

Pedro D. Llerenas
pedro.llerenas@cimat.mx

2025-11-20

Introducción

En este reporte, detallamos los métodos y propiedades de la clase `UniversalInteger`, una clase que nos centraliza la representación en hexadecimal y decimal. Además, se pueden realizar operaciones (suma y resta) con esta clase mediante la sobrecarga de operadores. Las representaciones correspondientes también son actualizadas con estas operaciones.

Definiciones

Antes de definir el `UniversalInteger`, definimos otras 2 clases por separado. Una es la `Hexadecimal`:

```
class Hexadecimal {
private:
    string x;
public:
    Hexadecimal(string a) {
        // implementation
    }

    Hexadecimal(int a) {
        // implementation
    }

    Hexadecimal operator+(Hexadecimal other) {
        // implementation
    }

    Hexadecimal operator-(Hexadecimal other) {
        // implementation
    }
};
```

y la clase `Binary`, que es esencialmente un *wrapper* de la clase `std::bitset`, con algunos comportamientos modificados.

```
class Binary {
private:
    bitset<32> bits;

public:
    Binary() {
        bits = bitset<32>(0);
    }
    Binary(bitset<32> i) {
        bits = i;
    }
    // ...
    Binary operator+(Binary other) {
        // implementation
    }
    Binary operator-(Binary other) {
        // implementation
    }
    Binary operator~() {
```

```
        // implementation
    }
}
```

Una vez definidas estas clases, las usamos para el `UniversalInteger` mediante composición.

```
class UniversalInteger {
private:
    int i;
    Binary b;
    Hexadecimal x;

public:
    UniversalInteger(string s) {
        // implementation
    }

    UniversalInteger operator+() {
        // implementation
    }
    UniversalInteger operator-() {
        // implementation
    }
};
```

Constructores

Los constructores de las clases `Binary`, `Hexadecimal` y `UniversalInteger` se encargan de convertir el argumento en una expresión normalizada. Por ejemplo, si realizamos

```
UniversalInteger("0x123")
```

los constructores garantizan que el formato sera de la forma `0x00000123`. Es decir, muestra los 32 bits. De igual manera, la clase `Binary` garantiza que el formato del numero serán los 32 bits. Usando el ejemplo anterior, tendríamos `0000000000000000000000000100100011`.

Podemos inicializar `UniversalInteger` con un string en formato binario, hexadecimal o decimal. Además, con un entero. El constructor sin parámetro simplemente asigna cada valor a su representación de 0.

Operaciones

Con las clases base ya definidas, la suma y resta de `UniversalInteger` simplemente debe utilizar el operador sobrecargado de `Binary` y `Hexadecimal` para asignar los valores correspondientes.

Compilación

Dado que la implementación de las clases se encuentran contenidas completamente en `integer.hpp`, solo debemos realizar

```
> g++ main.cpp -o main
```

Ejecución

Usar

```
> ./main
```

Una vez ejecutado el programa, se le pedirá al usuario ingresar una operación. En los ejemplos siguientes, primero insertamos `0xABC + 123`. Esto nos imprime la operación de cada una de las representaciones. En el segundo ejemplo, hacemos `0b10001010111 - 0x32A`.

```
> insert an operation '<a> <+|-> <b>'  
>  
> 0xABC + 123  
> int a: 2748 +  
> int b: 123  
> =====  
> int c: 2871  
>  
> bin a: 000000000000000000000101010111100 +  
> bin b: 00000000000000000000000000001111011  
> =====  
> bin c: 0000000000000000000001011100110111  
>  
> hex a: 0x00000ABC +  
> hex b: 0x0000007b  
> =====  
> hex c: 0x00000B37  
>  
>  
> insert an operation '<a> <+|-> <b>'  
>  
> 0b10001010111 - 0x32A  
> int a: 1111 -  
> int b: 810  
> =====  
> int c: 301  
>  
>  
> bin a: 000000000000000000000000010001010111 -  
> bin b: 00000000000000000000000001100101010  
> =====  
> bin c: 0000000000000000000000000100101101  
>  
>  
> hex a: 0x00000457 -  
> hex b: 0x0000032A  
> =====  
> hex c: 0x0000012D
```

Si intentamos insertar un operador invalido, el bucle continua, imprimiendo en la consola que el operador no fue valido.

Si intentamos insertar un numero con representación invalida (es decir, si el `string` no tiene representación binaria, hexadecimal o decimal), se toma como 0, y el programa realiza la operación como si insertáramos un 0.